
C: Como Programar

6ª Edição · Paul Deitel & Harvey Deitel

Capítulo 12

Estruturas de Dados em C

Exercícios (12.6–12.25)

Resolvidos passo a passo, com código completo

Contents

1	Exercício 12.6 – Concatenação de Listas	2
2	Exercício 12.7 – Mescla de Listas Ordenadas	4
3	Exercício 12.8 – Inserção Ordenada de 25 Inteiros	4
4	Exercício 12.9 – Lista Invertida	6
5	Exercício 12.10 – Inverter Linha de Texto	8
6	Exercício 12.11 – Testador de Palíndromos	9
7	Exercício 12.12 – Conversão Infixo → Pós-fixe	10
8	Exercício 12.13 – Avaliador Pós-fixe	12
9	Exercício 12.14 – Operandos com Múltiplos Dígitos	13
10	Exercício 12.15 – Simulação de Supermercado	15
11	Exercício 12.16 – Permitir Duplicatas	16
12	Exercício 12.17 – Árvore de Strings	17
13	Exercício 12.18 – Eliminação com Array vs Árvore	18
14	Exercício 12.19 – Profundidade da Árvore	19
15	Exercício 12.20 – Impressão Recursiva Reversa	19
16	Exercício 12.21 – Busca Recursiva	20
17	Exercício 12.22 – Exclusão em Árvore Binária	21
18	Exercício 12.23 – Busca em Árvore Binária	23
19	Exercício 12.24 – Travessia por Nível	23
20	Exercício 12.25 – Impressão Visual da Árvore	25

1. Exercício 12.6 – Concatenação de Listas

Resposta detalhada

```
1  /* Ex12.6: concatenar_listas.c */
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  typedef struct charNode {
6      char data;
7      struct charNode *nextPtr;
8  } CharNode;
9
10 void concatenate( CharNode **firstPtr, CharNode *
    secondPtr )
11 {
12     if ( *firstPtr == NULL ) {
13         *firstPtr = secondPtr;
14         return;
15     }
16     /* Avanca ate o ultimo no da primeira lista */
17     CharNode *p = *firstPtr;
18     while ( p->nextPtr != NULL ) {
19         p = p->nextPtr;
20     }
21     p->nextPtr = secondPtr; /* engata a segunda lista
        */
22 }
23
24 void inserir( CharNode **head, char c )
25 {
26     CharNode *novo = malloc( sizeof( CharNode ) );
27     novo->data = c;
28     novo->nextPtr = NULL;
29
30     if ( *head == NULL ) {
31         *head = novo;
32     } else {
33         CharNode *p = *head;
34         while ( p->nextPtr != NULL ) p = p->nextPtr;
35         p->nextPtr = novo;
36     }
37 }
38
39 void imprimir( CharNode *head )
40 {
41     while ( head != NULL ) {
42         printf( "%c ", head->data );
43         head = head->nextPtr;
44     }
```

```
45     printf( "\n" );
46 }
47
48 int main( void )
49 {
50     CharNode *l1 = NULL, *l2 = NULL;
51     inserir( &l1, 'A' );  inserir( &l1, 'B' );
52     inserir( &l1, 'C' );
53     inserir( &l2, 'X' );  inserir( &l2, 'Y' );
54     inserir( &l2, 'Z' );
55
56     printf( "L1: " );  imprimir( l1 );
57     printf( "L2: " );  imprimir( l2 );
58
59     concatenate( &l1, l2 );
60
61     printf( "Concatenada: " );  imprimir( l1 );
62
63     return 0;
64 }
```

Conceito-chave: concatenação em $O(n)$ – precisamos percorrer a primeira lista até o fim para então engatar a segunda. Listas duplamente encadeadas com ponteiro para o último nó tornariam isso $O(1)$.

2. Exercício 12.7 – Mescla de Listas Ordenadas

Resposta detalhada

```
1  /* Ex12.7: mesclar_ordenadas.c */
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  typedef struct intNode {
6      int data;
7      struct intNode *nextPtr;
8  } IntNode;
9
10 IntNode* merge( IntNode *a, IntNode *b )
11 {
12     IntNode dummy;          /* no sentinela para
13                             simplificar */
14     IntNode *tail = &dummy;
15     dummy.nextPtr = NULL;
16
17     while ( a != NULL && b != NULL ) {
18         if ( a->data <= b->data ) {
19             tail->nextPtr = a;
20             a = a->nextPtr;
21         } else {
22             tail->nextPtr = b;
23             b = b->nextPtr;
24         }
25         tail = tail->nextPtr;
26     }
27     /* Conecta o que sobrou */
28     tail->nextPtr = ( a != NULL ) ? a : b;
29
30     return dummy.nextPtr;
31 }
```

Algoritmo: compare as cabeças, escolha a menor, avance. Quando uma lista acabar, conecte a outra. $O(n + m)$.

Este é o “merge” do *merge sort* – uma das ideias mais elegantes da ciência da computação.

3. Exercício 12.8 – Inserção Ordenada de 25 Inteiros

Resposta detalhada

```
1  /* Ex12.8: insercao_ordenada.c */
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <time.h>
```

```
5
6 typedef struct node {
7     int data;
8     struct node *nextPtr;
9 } Node;
10
11 void inserirOrdenado( Node **head, int valor )
12 {
13     Node *novo = malloc( sizeof( Node ) );
14     novo->data = valor;
15
16     Node *prev = NULL, *cur = *head;
17     while ( cur != NULL && cur->data < valor ) {
18         prev = cur; cur = cur->nextPtr;
19     }
20     novo->nextPtr = cur;
21     if ( prev == NULL ) *head = novo;
22     else prev->nextPtr = novo;
23 }
24
25 int main( void )
26 {
27     Node *lista = NULL;
28     int i, valor, soma = 0;
29
30     srand( time( NULL ) );
31
32     for ( i = 0; i < 25; ++i ) {
33         valor = rand() % 101; /* 0 a 100 */
34         inserirOrdenado( &lista, valor );
35         soma += valor;
36     }
37
38     printf( "Lista ordenada:\n" );
39     Node *p = lista;
40     while ( p != NULL ) { printf( "%d ", p->data ); p
41                             = p->nextPtr; }
42     printf( "\nSoma:    %d\nMedia: %.2f\n", soma, soma
43           / 25.0 );
44
45     return 0;
46 }
```

4. Exercício 12.9 – Lista Invertida

Resposta detalhada

```
1  /* Ex12.9: inverter_lista.c
2     Cria lista de 10 chars e gera copia invertida */
3  #include <stdio.h>
4  #include <stdlib.h>
5
6  typedef struct cnode {
7      char data;
8      struct cnode *nextPtr;
9  } CNode;
10
11 void inserirFim( CNode **head, char c )
12 {
13     CNode *novo = malloc( sizeof( CNode ) );
14     novo->data = c; novo->nextPtr = NULL;
15     if ( *head == NULL ) { *head = novo; return; }
16     CNode *p = *head;
17     while ( p->nextPtr != NULL ) p = p->nextPtr;
18     p->nextPtr = novo;
19 }
20
21 /* Insere SEMPRE no inicio -> inverte naturalmente */
22 void inserirInicio( CNode **head, char c )
23 {
24     CNode *novo = malloc( sizeof( CNode ) );
25     novo->data = c;
26     novo->nextPtr = *head;
27     *head = novo;
28 }
29
30 int main( void )
31 {
32     CNode *original = NULL, *inversa = NULL;
33     char chars[] = "ABCDEFGHJIJ";
34     int i;
35
36     for ( i = 0; i < 10; ++i ) inserirFim( &original,
37                                             chars[ i ] );
38
39     /* Percorre original e insere no inicio da inversa
40     */
41     CNode *p = original;
42     while ( p != NULL ) {
43         inserirInicio( &inversa, p->data );
44         p = p->nextPtr;
45     }
```

```
45     printf( "Original: " );
46     p = original; while ( p != NULL ) { putchar( p->
        data ); p = p->nextPtr; }
47     printf( "\nInversa: " );
48     p = inversa; while ( p != NULL ) { putchar( p->
        data ); p = p->nextPtr; }
49     printf( "\n" );
50     return 0;
51 }
```

Truque elegante: ao percorrer a original do início ao fim e *inserir no início* da nova lista, obtemos a ordem invertida automaticamente.

5. Exercício 12.10 – Inverter Linha de Texto

Resposta detalhada

```
1  /* Ex12.10: inverter_texto.c
2     Usa pilha para inverter caracteres de uma linha */
3  #include <stdio.h>
4  #include <stdlib.h>
5
6  typedef struct snode {
7      char data;
8      struct snode *nextPtr;
9  } SNode;
10
11 void push( SNode **top, char c )
12 {
13     SNode *novo = malloc( sizeof( SNode ) );
14     novo->data = c;
15     novo->nextPtr = *top;
16     *top = novo;
17 }
18
19 char pop( SNode **top )
20 {
21     SNode *temp = *top;
22     char c = temp->data;
23     *top = temp->nextPtr;
24     free( temp );
25     return c;
26 }
27
28 int main( void )
29 {
30     SNode *pilha = NULL;
31     int c;
32
33     printf( "Digite uma linha: " );
34     while ( ( c = getchar() ) != '\n' && c != EOF ) {
35         push( &pilha, c );
36     }
37
38     printf( "Invertida: " );
39     while ( pilha != NULL ) {
40         putchar( pop( &pilha ) );
41     }
42     putchar( '\n' );
43     return 0;
44 }
```

Por que pilha funciona aqui? Empilhamos na ordem em que lemos e desempilhamos na ordem inversa – exatamente o que precisamos. LIFO em

ação.

6. Exercício 12.11 – Testador de Palíndromos

Resposta detalhada

Estratégia: usar uma pilha e uma fila simultaneamente. Cada caractere significativo (não-espaco, não-pontuação) é inserido em ambos. No fim, desempilhar e desenfileirar comparando – se diferentes em algum ponto, não é palíndromo.

```
1  /* Ex12.11: palindromo.c (esqueleto) */
2  #include <stdio.h>
3  #include <ctype.h>
4
5  /* (defina pilha e fila para char) */
6
7  int main( void )
8  {
9      SNode *pilha = NULL;
10     QNode *frenteFila = NULL, *finalFila = NULL;
11     int c;
12
13     printf( "Digite uma frase: " );
14     while ( ( c = getchar() ) != '\n' && c != EOF ) {
15         if ( isalpha( c ) ) {
16             char ch = tolower( c );
17             push( &pilha, ch );
18             enqueue( &frenteFila, &finalFila, ch );
19         }
20     }
21
22     int palindromo = 1;
23     while ( pilha != NULL && frenteFila != NULL ) {
24         if ( pop( &pilha ) != dequeue( &frenteFila ) )
25             {
26                 palindromo = 0;
27                 break;
28             }
29     }
30
31     printf( palindromo ? "Eh palindromo!\n"
32             : "Nao eh palindromo.\n" );
33     return 0;
34 }
```

Exemplos: “A man a plan a canal Panama”, “A base seba”, “Anotaram a data da maratona”. Espaços e pontuação são ignorados.

7. Exercício 12.12 – Conversão Infixo → Pós-fixado

Resposta detalhada

Algoritmo Shunting Yard (Dijkstra), versão simplificada para dígitos únicos:

```

1  /* Ex12.12: infix_to_postfix.c -- estrutura central */
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <string.h>
5
6  typedef struct stackNode {
7      char data;
8      struct stackNode *nextPtr;
9  } StackNode;
10 typedef StackNode *StackNodePtr;
11
12 void push( StackNodePtr *topPtr, char value );
13 char pop( StackNodePtr *topPtr );
14 char stackTop( StackNodePtr topPtr );
15 int isEmpty( StackNodePtr topPtr );
16 int isOperator( char c );
17 int precedence( char op1, char op2 );
18
19 void convertToPostfix( char infix[], char postfix[] )
20 {
21     StackNodePtr stack = NULL;
22     int infixIndex = 0, postfixIndex = 0;
23     char c;
24
25     push( &stack, '(' );           /* etapa 1 */
26     strcat( infix, ")" );         /* etapa 2 */
27
28     while ( !isEmpty( stack ) ) {
29         c = infix[ infixIndex ];
30
31         if ( c >= '0' && c <= '9' ) {
32             postfix[ postfixIndex++ ] = c;
33         }
34         else if ( c == '(' ) {
35             push( &stack, c );
36         }
37         else if ( isOperator( c ) ) {
38             while ( !isEmpty( stack ) &&
39                     isOperator( stackTop( stack ) ) &&
40                     precedence( stackTop( stack ), c )
41                         >= 0 ) {
42                 postfix[ postfixIndex++ ] = pop( &
43                     stack );
44             }
45         }
46     }
47 }

```

```

43         push( &stack, c );
44     }
45     else if ( c == ')' ) {
46         while ( stackTop( stack ) != '(' ) {
47             postfix[ postfixIndex++ ] = pop( &
48                 stack );
49         }
50         pop( &stack ); /* descarta '(' */
51     }
52     ++infixIndex;
53     postfix[ postfixIndex ] = '\0';
54 }
55
56 int isOperator( char c )
57 {
58     return c == '+' || c == '-' || c == '*' ||
59         c == '/' || c == '^' || c == '%';
60 }
61
62 int precedence( char op1, char op2 )
63 {
64     /* Retorna 1 se prec(op1) > prec(op2)
65        0 se iguais
66        -1 se prec(op1) < prec(op2) */
67     int p1, p2;
68     if ( op1 == '^' ) p1 = 3;
69     else if ( op1 == '*' || op1 == '/' ||
70         op1 == '%' ) p1 = 2;
71     else p1 = 1;
72     if ( op2 == '^' ) p2 = 3;
73     else if ( op2 == '*' || op2 == '/' ||
74         op2 == '%' ) p2 = 2;
75     else p2 = 1;
76     if ( p1 == p2 ) return 0;
77     return p1 > p2 ? 1 : -1;
78 }

```

Exemplo: $(6 + 2) * 5 - 8 / 4 \rightarrow 6 \ 2 + 5 * 8 \ 4 / -$

Por que pilha? Operadores precisam “esperar” seus operandos. A pilha guarda operadores até que a expressão esteja pronta para emití-los.

8. Exercício 12.13 – Avaliador Pós-fixado

Resposta detalhada

```
1  /* Ex12.13: postfix_eval.c */
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <string.h>
5  #include <math.h>
6
7  typedef struct stackNode {
8      int data;
9      struct stackNode *nextPtr;
10 } StackNode;
11 typedef StackNode *StackNodePtr;
12
13 void push( StackNodePtr *topPtr, int value );
14 int pop( StackNodePtr *topPtr );
15 int isEmpty( StackNodePtr topPtr );
16 int calculate( int op1, int op2, char op );
17
18 int evaluatePostfixExpression( char *expr )
19 {
20     StackNodePtr stack = NULL;
21     int i = 0;
22     char c;
23
24     strcat( expr, "\0" ); /* etapa 1 */
25
26     while ( ( c = expr[ i ] ) != '\0' ) {
27         if ( c >= '0' && c <= '9' ) {
28             push( &stack, c - '0' ); /* converte char
29                                     -> int */
30         }
31         else if ( c == '+' || c == '-' || c == '*' ||
32                 c == '/' || c == '^' || c == '%' ) {
33             int x = pop( &stack );
34             int y = pop( &stack );
35             push( &stack, calculate( y, x, c ) );
36         }
37         ++i;
38     }
39
40     return pop( &stack );
41 }
42
43 int calculate( int op1, int op2, char op )
44 {
45     switch ( op ) {
46         case '+': return op1 + op2;
```

```

46         case '-': return op1 - op2;
47         case '*': return op1 * op2;
48         case '/': return op1 / op2;
49         case '%': return op1 % op2;
50         case '^': return ( int )pow( op1, op2 );
51     }
52     return 0;
53 }

```

Trace de $6\ 2 + 5 * 8\ 4 / -$:

- a) $6 \rightarrow$ pilha [6]
- b) $2 \rightarrow$ pilha [6,2]
- c) $+$: pop 2, pop 6; push $6+2 \rightarrow$ [8]
- d) $5 \rightarrow$ [8,5]
- e) $*$: pop 5, pop 8; push $8*5 \rightarrow$ [40]
- f) $8 \rightarrow$ [40,8]
- g) $4 \rightarrow$ [40,8,4]
- h) $/$: pop 4, pop 8; push $8/4 \rightarrow$ [40,2]
- i) $-$: pop 2, pop 40; push $40-2 \rightarrow$ [38]

Resultado: **38**.

9. Exercício 12.14 – Operandos com Múltiplos Dígitos

Resposta detalhada

Modificação: agora os operandos podem ter vários dígitos. Como distinguir o fim de um número? Usar um *delimitador*, normalmente espaço.

```

1  /* Loop principal modificado */
2  i = 0;
3  while ( expr[ i ] != '\0' ) {
4      char c = expr[ i ];
5
6      if ( c >= '0' && c <= '9' ) {
7          int valor = 0;
8          /* Acumula dígitos enquanto for dígito */
9          while ( expr[ i ] >= '0' && expr[ i ] <= '9' )
10             {
11                 valor = valor * 10 + ( expr[ i ] - '0' );
12                 ++i;
13             }
14             push( &stack, valor );
15             continue; /* nao incrementa i de novo */
16         }
17         /* ... resto igual ... */
18         ++i;
19     }

```

Princípio: construir o número dígito a dígito multiplicando o acumulador

por 10. Esta é a base de todos os parsers numéricos – de `atoi` aos *lexers* de compiladores.

10. Exercício 12.15 – Simulação de Supermercado

Resposta detalhada

```
1  /* Ex12.15: supermercado.c -- esqueleto */
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <time.h>
5
6  typedef struct cliente {
7      int chegada;          /* minuto em que chegou na
8                          fila */
9      struct cliente *next;
10 } Cliente;
11
12 Cliente *cabeca = NULL, *cauda = NULL;
13
14 void enqueue( int t );
15 int dequeue( void );
16 int tamanhoFila( void );
17
18 int main( void )
19 {
20     srand( time( NULL ) );
21
22     int prox_chegada = 1 + rand() % 4;
23     int caixa_livre_em = 0;
24     int max_fila = 0, max_espera = 0;
25
26     int t;
27     for ( t = 1; t <= 720; ++t ) {
28         /* Chegada */
29         if ( t == prox_chegada ) {
30             enqueue( t );
31             int sz = tamanhoFila();
32             if ( sz > max_fila ) max_fila = sz;
33             prox_chegada = t + 1 + rand() % 4;
34         }
35         /* Caixa livre? */
36         if ( t >= caixa_livre_em && cabeca != NULL ) {
37             int chegou = dequeue();
38             int espera = t - chegou;
39             if ( espera > max_espera ) max_espera =
40                 espera;
41             caixa_livre_em = t + 1 + rand() % 4;
42         }
43     }
44
45     printf( "Maxima fila:  %d\n", max_fila );
46     printf( "Espera maxima: %d minutos\n", max_espera );
47 }
```



```
    );  
45     return 0;  
46 }
```

Respostas típicas (variam por aleatoriedade):

- (a) **Fila máxima:** tipicamente 2–5 clientes.
- (b) **Espera máxima:** tipicamente 5–15 minutos.
- (c) **Se chegada mudar para 1–3 min:** taxa de chegada (~2 min/cliente) passa a ser menor que a de atendimento (~2.5 min/cliente), causando **crescimento sem limites** da fila ao longo do dia.

Lição: este é um exemplo de *teoria das filas* – ramo da matemática aplicada à engenharia de tráfego, redes, call centers, etc.

11. Exercício 12.16 – Permitir Duplicatas

Resposta detalhada

Modificação simples: na função `insertNode`, alterar a condição de comparação para aceitar iguais em um dos lados:

```
1  /* Original: descarta duplicatas  
2     if ( valor < no->data ) ... esquerda  
3     else if ( valor > no->data ) ... direita  
4     else ; // duplicata -- ignora  
5  
6     Modificado: aceita duplicatas, vai para a direita:  
7     */  
7  if ( valor < no->data ) {  
8     /* insere a esquerda */  
9  } else { /* valor >= no->data */  
10     /* insere a direita */  
11 }
```

Consequência: a travessia in-order ainda produz ordem crescente, mas com duplicatas adjacentes. Outras operações (busca, exclusão) funcionam normalmente.

12. Exercício 12.17 – Árvore de Strings

Resposta detalhada

```
1  /* Ex12.17: arvore_strings.c -- esqueleto */
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <string.h>
5
6  typedef struct treeNode {
7      char *str;
8      struct treeNode *left, *right;
9  } TreeNode;
10
11 void insert( TreeNode **root, char *palavra )
12 {
13     if ( *root == NULL ) {
14         TreeNode *novo = malloc( sizeof( TreeNode ) );
15         novo->str = malloc( strlen( palavra ) + 1 );
16         strcpy( novo->str, palavra );
17         novo->left = novo->right = NULL;
18         *root = novo;
19         return;
20     }
21     int cmp = strcmp( palavra, (*root)->str );
22     if ( cmp < 0 ) insert( &(*root)->left,
23         palavra );
24     else if ( cmp > 0 ) insert( &(*root)->right,
25         palavra );
26     /* iguais: ignora (sem duplicatas) */
27 }
28
29 int main( void )
30 {
31     TreeNode *raiz = NULL;
32     char texto[ 1000 ];
33     char *tok;
34
35     printf( "Digite uma linha de texto:\n" );
36     fgets( texto, 1000, stdin );
37
38     tok = strtok( texto, " \t\n.,;:!?\" );
39     while ( tok != NULL ) {
40         insert( &raiz, tok );
41         tok = strtok( NULL, " \t\n.,;:!?\" );
42     }
43
44     /* Chamar inOrder, preOrder, postOrder */
45     return 0;
46 }
```

Nota: a travessia in-order da árvore produz as palavras em ordem alfabética – excelente para criar índices de livros ou dicionários.

13. Exercício 12.18 – Eliminação com Array vs Árvore

Resposta detalhada

Eliminação com array:

- a) Para cada novo valor, percorrer todo o array procurando duplicata.
- b) Se não encontrar, inserir no final.
- c) Complexidade: $O(n)$ por inserção; total $O(n^2)$ para n valores.

Eliminação com árvore binária de busca:

- a) Para cada novo valor, descer pela árvore.
- b) Se chegar a um nó vazio: inserir.
- c) Se encontrar igual: ignorar.
- d) Complexidade média: $O(\log n)$ por inserção; total $O(n \log n)$.
- e) Pior caso (árvore desbalanceada, dados já ordenados): $O(n^2)$.

Comparação:

Estrutura	Médio	Pior caso
Array	$O(n^2)$	$O(n^2)$
Árvore BST	$O(n \log n)$	$O(n^2)$
Árvore balanceada (AVL, RB)	$O(n \log n)$	$O(n \log n)$
Tabela hash	$O(n)$	$O(n^2)$ raro

Para grandes conjuntos de dados, árvores balanceadas ou tabelas hash são as escolhas profissionais.

14. Exercício 12.19 – Profundidade da Árvore

Resposta detalhada

```
1  /* Ex12.19: profundidade.c */
2  int depth( const TreeNode *root )
3  {
4      if ( root == NULL ) return 0;
5
6      int d_esq = depth( root->left );
7      int d_dir = depth( root->right );
8
9      return 1 + ( d_esq > d_dir ? d_esq : d_dir );
10 }
```

Análise:

- Caso base: árvore vazia tem profundidade 0.
- Recursão: profundidade = 1 (este nó) + máximo das profundidades das subárvores.

Para a árvore do Ex. 12.5: a profundidade é 4 níveis (raiz 49, depois 28/83, depois 18/40/71/97, e por fim 11/19/32/44/69/72/92/99).

15. Exercício 12.20 – Impressão Recursiva Reversa

Resposta detalhada

```
1  /* Ex12.20: print_reverse.c */
2  void printListBackwards( const Node *p )
3  {
4      if ( p == NULL ) return;
5
6      printListBackwards( p->nextPtr ); /* chama
7                                         primeiro */
8      printf( "%d ", p->data );          /* imprime
9                                         depois */
10 }
```

Por que funciona? Cada chamada recursiva mergulha mais fundo na lista *antes* de imprimir. Quando chega ao fim, retorna – e a impressão acontece na ordem reversa do empilhamento de chamadas. A pilha de chamadas faz o trabalho da pilha explícita do Ex. 12.10.

Cuidado: para listas muito longas, isso pode estourar a pilha de chamadas (*stack overflow*). Para essas, prefira a versão iterativa com pilha explícita.

16. Exercício 12.21 – Busca Recursiva

Resposta detalhada

```
1  /* Ex12.21: search_recursive.c */
2  Node* searchList( Node *head, int valor )
3  {
4      if ( head == NULL )          return NULL;
5      if ( head->data == valor )    return head;
6      return searchList( head->nextPtr, valor );
7  }
```

Três casos:

- a) Lista vazia: retorna NULL (não achou).
- b) Achou: retorna o ponteiro para o nó.
- c) Senão: busca recursivamente no resto.

Padrão: esta é uma busca em lista em $O(n)$ – mesma complexidade da versão iterativa, mas usa pilha de chamadas. Tail-recursive: bons compiladores otimizam para um loop interno.

17. Exercício 12.22 – Exclusão em Árvore Binária

Resposta detalhada

Três casos da exclusão:

Caso 1 – Nó folha: Simplesmente remove o nó e ajusta o pai para NULL.

Caso 2 – Nó com um filho: Substitui o nó pelo seu único filho.

Caso 3 – Nó com dois filhos: Procura o *substituto* (predecessor in-order: o maior valor da subárvore esquerda). Copia o valor do substituto para o nó atual e remove o substituto (que é caso 1 ou 2, mais simples).

```
1  /* Ex12.22: delete_bst.c -- estrutura central */
2  TreeNode* deleteNode( TreeNode *root, int valor )
3  {
4      if ( root == NULL ) {
5          printf( "Valor %d nao encontrado.\n", valor );
6          return NULL;
7      }
8
9      if ( valor < root->data ) {
10         root->left = deleteNode( root->left, valor );
11     } else if ( valor > root->data ) {
12         root->right = deleteNode( root->right, valor );
13     } else {
14         /* Achou o no a remover */
15
16         if ( root->left == NULL && root->right == NULL ) {
17             /* Caso 1: folha */
18             free( root );
19             return NULL;
20         }
21         else if ( root->left == NULL ) {
22             /* Caso 2a: so filho direito */
23             TreeNode *temp = root->right;
24             free( root );
25             return temp;
26         }
27         else if ( root->right == NULL ) {
28             /* Caso 2b: so filho esquerdo */
29             TreeNode *temp = root->left;
30             free( root );
31             return temp;
32         }
33         else {
34             /* Caso 3: dois filhos. Procura substituto
35              :
36              maior valor da subarvore esquerda */
37             TreeNode *sub = root->left;
38             while ( sub->right != NULL ) sub = sub->
```

```

38         right;
39         root->data = sub->data;
40         root->left = deleteNode( root->left, sub->
41             data );
42     }
43     return root;
44 }
```

Observação: o uso de recursão e retorno de `TreeNode *` simplifica drasticamente o código – evita rastreamento manual do nó pai. Padrão idiomático em todas as operações em árvores.

18. Exercício 12.23 – Busca em Árvore Binária

Resposta detalhada

```
1  /* Ex12.23: bst_search.c */
2  TreeNode* binaryTreeSearch( TreeNode *root, int chave
3  )
4  {
5      if ( root == NULL )           return NULL;
6      if ( chave == root->data )    return root;
7      if ( chave < root->data )
8          return binaryTreeSearch( root->left,  chave );
9      return binaryTreeSearch( root->right, chave );
10 }
```

Por que árvores são eficientes para busca? A cada passo, descartamos *metade* da árvore restante: $O(\log n)$ em média. Para um milhão de elementos, isso significa ~ 20 comparações, em vez de $\sim 500\,000$ em uma lista.

Esta é a base de `std::map` (C++), `TreeMap` (Java), índices B-tree em bancos de dados, etc.

19. Exercício 12.24 – Travessia por Nível

Resposta detalhada

Algoritmo BFS (Breadth-First Search) com fila:

```
1  /* Ex12.24: level_order.c */
2  void levelOrder( TreeNode *root )
3  {
4      if ( root == NULL ) return;
5
6      /* Fila de ponteiros para no */
7      TreeNode *fila[ 1000 ];
8      int frente = 0, fim = 0;
9
10     fila[ fim++ ] = root;
11
12     while ( frente < fim ) {
13         TreeNode *atual = fila[ frente++ ];
14         printf( "%d ", atual->data );
15
16         if ( atual->left != NULL ) fila[ fim++ ] =
17             atual->left;
18         if ( atual->right != NULL ) fila[ fim++ ] =
19             atual->right;
20     }
21 }
```

Para a árvore do Ex. 12.5, a travessia por nível é:


```
1 49 28 83 18 40 71 97 11 19 32 44 69 72 92 99
```

Aplicações de BFS:

- Encontrar caminho mais curto em grafos não-ponderados.
- Web crawling em largura.
- Renderização de árvores de UI.
- Algoritmos de inteligência artificial (jogos, planejamento).

20. Exercício 12.25 – Impressão Visual da Árvore

Resposta detalhada

Truque: é uma travessia in-order *modificada*: visite primeiro o filho **direito**, depois imprima o nó atual, depois visite o filho esquerdo. Como gira a árvore 90° à esquerda, a saída visual é a árvore deitada com a raiz à esquerda.

```
1  /* Ex12.25: outputTree.c */
2  void outputTree( const TreeNode *root, int totalSpaces
3  )
4  {
5      if ( root == NULL ) return;
6
7      /* Visita direita PRIMEIRO (vai para topo da saída
8       ) */
9      outputTree( root->right, totalSpaces + 5 );
10
11     /* Espacos antes do valor */
12     int i;
13     for ( i = 1; i <= totalSpaces; ++i ) putchar( ' ',
14     );
15
16     /* Imprime o nó */
17     printf( "%d\n", root->data );
18
19     /* Visita esquerda DEPOIS (vai para baixo da saída
20      ) */
21     outputTree( root->left, totalSpaces + 5 );
22 }
```

Para a árvore do Ex. 12.5, a saída seria:

Saída do programa

```

                                     99
97
92
83
72
71
69
49
44
40
32
28
19
18
11
```

Por que isso funciona? A travessia in-order produz a ordem crescente

(esquerda → nó → direita). Trocando a ordem para direita → nó → esquerda, produzimos ordem decrescente – exatamente o que precisamos para exibir a árvore com os maiores no topo. O recuo (`totalSpaces`) cresce com a profundidade, dando o efeito 3D.

Aplicações: esta técnica é a base de visualização de árvores em ferramentas de *debugging*, editores de XML/JSON, e até no comando Unix `tree`.

Exercícios 12.26 a 12.30

Resposta detalhada

Os exercícios 12.26 a 12.30 (construção de um compilador para a linguagem Simple, gerando código SML para o Simpletron) são projetos extensos disponibilizados em PDF separado pela Deitel & Associates. Eles consolidam:

- **12.26:** *lexer* e *parser* para a linguagem Simple
- **12.27:** análise sintática e geração de código SML
- **12.28-30:** otimizações, recursos avançados, testes

Sugerimos consultar o documento original no site da Deitel para os enunciados completos. Estes exercícios são *projetos de final de curso* e exigem várias semanas de trabalho.